

AVL Tree

1 Introduction

An AVL Tree is a self-balancing Binary Search Tree (BST) in which the height difference between the left and right subtrees of any node is at most 1.

It was introduced by **Adelson-Velsky and Landis**.

2 Balance Factor

The balance factor (BF) of a node is defined as:

$$BF = \text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree})$$

$$BF \in \{-1, 0, +1\}$$

If the balance factor violates this condition, rotations are applied.

3 AVL Tree Rotations

3.1 LL Rotation (Left-Left)

Occurs when a node becomes unbalanced due to insertion in the left subtree of the left child.

3.2 RR Rotation (Right-Right)

Occurs when a node becomes unbalanced due to insertion in the right subtree of the right child.

3.3 LR Rotation (Left-Right)

Occurs when insertion is in the right subtree of the left child.

3.4 RL Rotation (Right-Left)

Occurs when insertion is in the left subtree of the right child.

4 AVL Tree Insertion Example

Insert the following elements:

30, 20, 10, 25, 40, 50

Balance factor is shown as **value(BF)**.

Step 1: Insert 30

30(0)

Step 2: Insert 20

30(+1)
/

20(0)

Step 3: Insert 10 (LL Rotation)

Before Rotation

30(+2)
/

20(+1)
/

10(0)

After LL Rotation

20(0)
/ \

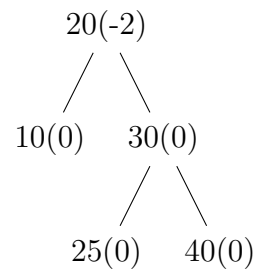
10(0) 30(0)

Step 4: Insert 25

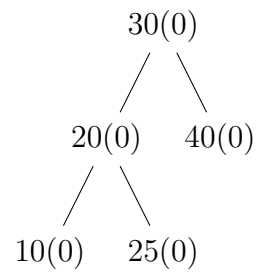
20(-1)
/ \

10(0) 30(+1)
/

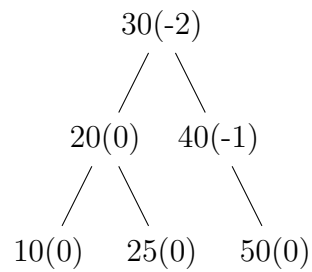
25(0)

Step 5: Insert 40

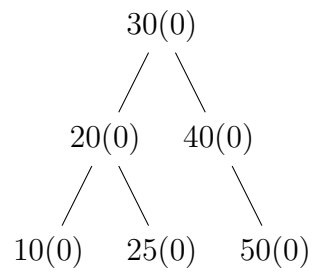
RR Rotation Required at Node 20

**Step 6: Insert 50 (RR Rotation)**

Before Rotation



After RR Rotation



5 AVL Insertion Algorithm

Algorithm 1 AVL Insert

```

1: function INSERT(node, key)
2:   if node is NULL then
3:     return new node
4:   end if
5:   if key < node.key then
6:     node.left ← Insert(node.left, key)
7:   else if key > node.key then
8:     node.right ← Insert(node.right, key)
9:   end if
10:  Update height of node
11:  Compute balance factor
12:  if BF > 1 and key < node.left.key then
13:    Perform LL Rotation
14:  else if BF < -1 and key > node.right.key then
15:    Perform RR Rotation
16:  else if BF > 1 and key > node.left.key then
17:    Perform LR Rotation
18:  else if BF < -1 and key < node.right.key then
19:    Perform RL Rotation
20:  end if
21:  return node
22: end function

```

6 Time and Space Complexity

- Time Complexity: $O(\log n)$
- Space Complexity: $O(n)$

7 Conclusion

AVL Trees guarantee balanced height, ensuring efficient searching, insertion, and deletion operations. Rotations maintain balance while preserving BST properties.